

Configuração Inicial da Rede Social

<https://gemini.google.com/app/60f36e415ce7fab1>

User prompt: considere que você é um engenheiro de software senior. organize as instruções da atividade presente "AP1 — Desenvolvimento de Software Apoiado por IA", separando em pequenos passos e me guiando durante toda a execução desse projeto. tomarei as decisões com base nas suas informações e instruções. o projeto escolhido sera uma rede social usando o twitter como base. vamos fazer cada tarefa especifica por vez

Response: Olá! Como engenheiro de software sênior, serei seu mentor técnico para garantir que o seu projeto seja um sucesso e atenda a todos os requisitos. Um clone do Twitter é uma excelente escolha para esta atividade, pois é uma aplicação não trivial. Como a avaliação exige no mínimo 100 mil linhas de código (contadas com a ferramenta cloc), ter uma aplicação com diversas interações sociais nos dará bastante escopo.

O nosso fluxo de trabalho seguirá estritamente a metodologia *Specification-Driven Development* (SDD). Isso significa que o nosso lema será "especificar antes de implementar". A especificação será a nossa fonte de verdade e o contrato para o agente de IA.

O prazo é apertado: a apresentação será no dia 01/10 e o último commit válido que contará para a entrega deve ser feito até as 18h desse dia.

Como combinamos de avançar um passo por vez, dividi o trabalho nas seguintes grandes fases, mas hoje focaremos **apenas** na primeira:

1. **Configuração Inicial** (Repositório e Estruturas)
2. **Especificação (SDD)** (Escrever o que vamos construir)
3. **Planejamento e Implementação** (Uso dos agentes de IA, logs de prompts e commits)
4. **Validação** (Testes e checagem de requisitos)
5. **Deploy e Entrega** (Publicação da URL pública e preparação da apresentação)

Passo 1: Configuração do Repositório e Estrutura

Sua primeira missão é configurar o ambiente base corretamente, pois a organização é critério de avaliação. Siga os passos abaixo no seu ambiente de desenvolvimento:

1. **Criação do Repositório:**
 - Crie um repositório novo e público no GitHub.
 - Este repositório deve ser separado do seu diário.
 - Sugestão de nome: dsai-ap1-twitter (seguindo a recomendação da regra dsai-ap1-<nome-da-app>).
2. **Regras de Envio (Commit):**

- Todo código deve subir exclusivamente via git commit e git push.
- É estritamente proibido fazer upload de arquivos pela interface web do GitHub.
- Você e a sua dupla devem fazer os commits logados com as próprias contas, pois é por meio do autor do commit que a condução de cada sessão será avaliada.

3. **Estrutura de Pastas Obrigatória:** Crie exatamente esta estrutura na raiz do seu projeto:

- README.md (Arquivo que futuramente conterá a URL do projeto, o nome da dupla, a stack, como rodar, ferramentas, modelos usados e a saída do cloc).
- SPEC/ (Pasta onde colocaremos uma especificação datada para cada parte do sistema).
- src/ (Pasta para o código-fonte, a organização interna aqui é livre).
- tests/ (Pasta para os testes).
- prompts/sessoes/ (Pasta onde ficarão as exportações brutas de todas as ferramentas e prompts utilizados).

O que você deve fazer agora: Abra o seu terminal, crie o repositório, faça o clone, monte as pastas e arquivos vazios listados acima e faça o seu primeiro *commit* inicial (ex: "Initial commit").

Quando terminar, responda com "Pronto" e **me informe qual stack de desenvolvimento (Linguagens/Frameworks) você deseja usar** no frontend e backend. Assim que fizer isso, te passarei as instruções do Passo 2 para criarmos a nossa primeira especificação (SDD) na pasta SPEC/!

User prompt: seria melhor mudar o nome do app para algo original mas simples. alem disso, acho que as pastas devem ser criadas a medidas que os arquivos forem gerados

Response: Faz total sentido técnico: o Git sequer rastreia diretórios vazios por padrão, então criar as pastas no momento em que seus respectivos artefatos forem escritos mantém o repositório limpo e alinhado ao fluxo do SDD.

Para o nome da aplicação, a recomendação é seguir a convenção de nomenclatura do repositório dsai-ap1-<nome-da-app>. Ideias curtas, diretas e com identidade própria de microblogging:

- **Chirp** (dsai-ap1-chirp): clássico, remete a posts rápidos e curtos.
- **Pulso** (dsai-ap1-pulso): dinâmico, focado na ideia de acompanhar o ritmo dos acontecimentos em tempo real.
- **Echo** (dsai-ap1-echo): direto, associado à propagação de mensagens e reposts.
- **Murmur** (dsai-ap1-murmur): simples, moderno e minimalista.
- **Piu** (dsai-ap1-piu): irreverente, brasileiro e bem descritivo para um clone da rede do pássaro.

Com a estratégia de criar as pastas sob demanda, o nosso fluxo inicial fica enxuto:

1. **Repositório e Raiz:** Criação do repositório no GitHub (dsai-ap1-<nome-escolhido>). O primeiro arquivo versionado na raiz será apenas o README.md inicial.
2. **Primeira pasta (SPEC/):** Antes de qualquer código, criaremos a pasta SPEC/ com o documento fundacional datado (por exemplo, SPEC/2026-09-30-visao-geral.md). Este commit entra obrigatoriamente antes de qualquer pasta src/ ou tests/.
3. **Registro de prompts (prompts/sessoes/):** Criada assim que concluirmos e exportarmos a primeira sessão com a IA.

Vale ressaltar um ponto de atenção técnica sobre a métrica de 100 mil linhas de código (cloc) sem contar

arquivos de configuração, build, dependências ou JSON: a stack escolhida precisará ser bem estruturada para suportar um ecossistema completo de backend, frontend e suíte de testes robusta gerada pelo agente.

Qual dos nomes você escolhe para o projeto e qual stack técnica prefere utilizar no backend e frontend?

User prompt: o nome por ser echo, e a stack sera python

Response: Repositório dsai-ap1-echo definido. Para aproveitar o seu ambiente de desenvolvimento no Linux com VS Code e a sua familiaridade com estruturação de consultas SQL, utilizar frameworks robustos como Django ou FastAPI para o backend será uma excelente base.

Vamos iniciar a fase de **Especificação (SDD)**. O princípio central aqui é que a especificação é a fonte de verdade, e deve separar a intenção dos detalhes de implementação para reduzir ambiguidades. O fluxo do projeto determina estritamente que cada especificação deve ser escrita e salva em um commit *antes* do código que ela descreve.

Sua tarefa agora é criar o documento de visão geral do Echo.

Instruções para este passo:

1. **Crie o arquivo:** Na raiz do projeto, crie a pasta SPEC/ e dentro dela o arquivo 2026-09-30-visao-geral.md. A regra de nomenclatura exigida é AAAA-MM-DD-<parte>.md.
2. **Estruture o conteúdo:** O documento deve conter as três seções obrigatórias para guiar a IA:
 - **O quê e por quê:** A descrição geral do nosso clone do Twitter (Echo) e seu propósito.
 - **Critérios de aceitação:** Regras diretas e verificáveis que o agente usará para saber quando parar de gerar código (ex: limite de caracteres, timeline cronológica, persistência no banco).
 - **Fora do escopo:** Funcionalidades que o sistema *não* fará nesta etapa (ex: mensagens diretas, upload de vídeos).
3. **Padrão de Commit:** Ao commitar este arquivo, a mensagem precisa obrigatoriamente terminar com dois trailers declarando a origem. Como você escreverá essa spec, a formatação exata no terminal deve ser:

```
git commit -m "docs: define visao geral do escopo do echo"
```

Agent: none

Spec: SPEC/2026-09-30-visao-geral.md"

Escreva um rascunho do conteúdo do arquivo 2026-09-30-visao-geral.md e cole aqui. Faremos uma rápida revisão para garantir que os critérios de aceitação não deixam ambiguidades para a IA antes de você realizar o commit e avançarmos para a primeira sessão de implementação.

User prompt: analise o texto da spec, e faça correções necessárias. O Echo é uma rede social de posts curtos inspirada no Twitter. Nele uma pessoa cria conta, entra e sai. Publica posts que podem ter texto, imagens ou os dois juntos. O texto do post tem no maximo 280 caracteres. Cada post aceita ate 4 imagens nos formatos JPEG, PNG ou WebP, com no maximo 5MB cada. A linha do tempo é cronologica reversa,

sem algoritmo de recomendacao. O usuario ve primeiro os posts de quem ele segue e tambem os proprios. Cada perfil tem uma pagina publica que lista os posts daquele usuario. O usuario pode curtir, responder e repostar, sem duplicar essas acoes. Pode seguir e deixar de seguir outras pessoas, mas nao pode seguir a si mesmo. Os dados persistem em banco, e senhas nunca ficam em texto puro. A aplicacao precisa ficar acessivel em URL publica, abrindo com um clique. E o repositorio precisa somar no minimo 100 mil linhas de codigo, contadas com cloc. O que o sistema nao deve ter: mensagens diretas, lives, enquetes, espacos de audio, algoritmo de recomendacao, anuncios, verificacao paga, notificacoes push nem aplicativo mobile nativo. Estruture isso num arquivo markdown com tres secoes. A primeira se chama O que e por que e deve ter um paragrafo curto explicando o produto. A segunda se chama Critérios de aceitacao e deve ter bullets verificaveis, agrupados por tema: conta, posts, midia, linha do tempo, interacoes, relacoes sociais, dados e entrega. A terceira se chama Fora do escopo e lista o que nao vai ser feito. Algumas regras para o texto. Nao cite framework, banco nem biblioteca, isso fica para outra etapa. Cada criterio precisa ser testavel, sem adjetivos vagos como rapido, bonito ou moderno. Inclua casos limite como post vazio, tentar publicar 281 caracteres, tentar editar post de outra pessoa. Me devolva o markdown completo no chat, pronto para eu copiar e salvar como SPEC/2026-09-30-visao-geral.md.

Response: O texto da sua especificação está excelente e perfeitamente alinhado com o princípio de separar a intenção da implementação técnica. A estrutura que adotou, com as secções "O quê e por quê", "Critérios de aceitação" e "Fora do escopo", segue exatamente o formato exigido para as especificações desta atividade prática. A inclusão das restrições de entrega (mínimo de 100 mil linhas contadas com a ferramenta cloc e a exigência de uma URL pública acessível com um clique) diretamente nos critérios de aceitação é uma ótima estratégia para garantir que o agente de IA não desconsidere estes requisitos de avaliação.

Fiz as devidas correções e ajustes solicitados. Removi qualquer alusão a ferramentas de backend ou infraestrutura, reescrevi os pontos de forma testável e adicionei os casos limite (como submissões com 281 caracteres, posts vazios, limite de ficheiros e proteção contra a edição de publicações alheias).

Aqui está o conteúdo em formato Markdown. Pode copiá-lo e guardá-lo na sua pasta SPEC/ como 2026-09-30-visao-geral.md:

```
# Echo (2026-09-30)
```

```
## O quê e por quê
```

O Echo é uma rede social de microblogging público focada em publicações curtas, inspirada no Twitter. O objetivo do sistema é permitir que os utilizadores partilhem ideias de forma rápida, acompanhem as atualizações de outras pessoas em tempo real através de uma linha do tempo cronológica e interajam por meio de curtidas, repostagens e respostas.

```
## Critérios de aceitação
```

```
**Conta**
```

- Um utilizador deve poder criar uma conta, iniciar sessão (login) e terminar sessão (logout).
- O sistema deve rejeitar a criação de uma conta se o nome de utilizador ou o email já estiverem registados.

```
**Posts**
```

- Uma publicação (post) pode conter texto, imagens ou ambos.
- O texto de um post tem o limite estrito de 280 caracteres.

- O sistema deve falhar e rejeitar a publicação se o texto submetido contiver 281 caracteres ou mais.
- O sistema deve rejeitar um post que seja enviado completamente vazio (sem qualquer texto e sem imagens anexadas).
- Apenas o autor original pode apagar o seu próprio post. O sistema deve bloquear qualquer tentativa de um utilizador apagar ou editar posts de outras pessoas.
- Uma vez publicado, um post não pode ser editado (apenas apagado).

****Mídia****

- Um post pode conter até 4 imagens associadas.
- O sistema deve rejeitar ficheiros que não sejam imagens, aceitando exclusivamente os formatos JPEG, PNG ou WebP.
- O sistema deve rejeitar o upload se uma única imagem ultrapassar o limite de 5MB de tamanho.
- O sistema deve bloquear a publicação se o utilizador tentar anexar 5 ou mais imagens num único post.

****Linha do tempo****

- A página inicial (linha do tempo) deve exibir os posts em ordem cronológica reversa (do mais recente para o mais antigo).
- A linha do tempo deve exibir exclusivamente os posts dos utilizadores que a pessoa segue, juntamente com os seus próprios posts.
- A página de perfil público de um utilizador deve listar apenas os posts criados por esse utilizador específico.

****Interações****

- O utilizador pode curtir, responder e repostar uma publicação.
- O sistema deve impedir a duplicação destas ações pelo mesmo utilizador no mesmo post (por exemplo, ao clicar em "curtir" novamente num post já curtido, a curtida deve ser removida em vez de somada).

****Relações sociais****

- O utilizador pode seguir e deixar de seguir outros perfis públicos.
- O sistema deve bloquear qualquer tentativa de um utilizador seguir a si mesmo.

****Dados****

- Todos os dados gerados (utilizadores, posts, relações, interações) devem ser persistidos numa base de dados relacional.
- As senhas dos utilizadores nunca devem ser guardadas em texto puro, devendo ser protegidas de forma irreversível antes da persistência.

****Entrega****

- A aplicação final deve estar publicada e ser acessível através de uma URL pública, abrindo imediatamente com um clique.
- O código-fonte produzido no repositório (excluindo dependências, dados e ficheiros gerados automaticamente) deve somar no mínimo 100 mil linhas de código (LOC), verificadas estritamente com a ferramenta `cloc`.

Fora do escopo

- Mensagens diretas (chat privado).

- Transmissões ao vivo (lives).
- Enquetes / Sondagens.
- Espaços de áudio.
- Algoritmos de recomendação na linha do tempo.
- Anúncios ou sistemas de monetização.
- Selos de verificação paga.
- Notificações push.
- Aplicativo mobile nativo (o projeto terá apenas interface web).

Lembre-se de realizar o *commit* deste ficheiro utilizando exatamente a formatação com os *trailers* exigidos antes de gerar o código associado a ele:

```
git commit -m "docs: define visao geral do escopo do echo"
```

Agent: none

Spec: SPEC/2026-09-30-visao-geral.md"

Assim que guardar o ficheiro e efetuar este *commit*, avise-me para iniciarmos a próxima fase, onde iremos configurar o agente de codificação de IA e criar o planeamento técnico da nossa *stack* Python.
